

React Lab: Unit Testing and Mocking with Jest

Hands-on Lab Solution — "gitclientapp" (GitClient module, App component, and mocked axios tests)

Objectives

- Understand the need for isolation in testing
- Understand the concept of mocking
- Use Jest for unit testing and mocking

Lab Goals

- Unit test modules in isolation
- Create and configure mocks and spies

Prerequisites

- Node.js
- NPM
- Visual Studio Code

1. The Need for Isolation in Testing

A unit test should exercise a single unit of code — a function, class, or module — on its own, in isolation from the other things it depends on, such as a network API, a database, the file system, or the current date/time. In this lab, GitClient depends on axios to make an HTTP call to api.github.com. If a test for GitClient were allowed to make that real network call, the test would suffer from several problems:

- Slow — real HTTP requests are orders of magnitude slower than in-memory function calls, and slow tests get skipped or run less often.
- Flaky — the test's outcome now depends on network connectivity, GitHub's uptime, and rate limits, none of which the test itself controls.
- Non-deterministic — the real GitHub API can return different repositories tomorrow than it does today, so assertions about exact values become unreliable.
- Unclear failures — if the test fails, it's hard to tell whether the bug is in GitClient's own logic or in GitHub's API, the network, or axios itself.

Isolating the unit under test from its dependencies fixes all of this: the test becomes fast, deterministic, and repeatable, and a failure can only mean one thing — the code being tested is behaving incorrectly.

2. The Concept of Mocking

Mocking means replacing a real dependency with a fake stand-in ('mock' or 'test double') that mimics the dependency's interface but returns controlled, predictable data instead of performing the real operation. In this lab, axios.get is mocked so that, instead of sending an actual HTTP request, it immediately resolves with a hand-written, dummy list of repositories. This lets the test verify that GitClient correctly processes whatever axios gives it, without caring how axios itself works or whether the real API is reachable.

Spy	Mock
Wraps a real function and records how it was called (arguments, call count), usually while still letting the real implementation run.	Fully replaces a function's implementation, returning a fixed value or fake behavior instead of the real one.
Used to verify a function was called correctly, without changing what it does.	Used to isolate the code under test from a dependency's real behavior (e.g. a real network call).
<code>jest.spyOn(object, 'method')</code>	<code>jest.mock('module')</code> or <code>jest.fn()</code>

3. Using Jest for Unit Testing and Mocking

Jest is the test runner and assertion library that ships by default with Create React App projects. Tests are grouped with `describe()` and defined individually with `test()` (or its alias, `it()`). Inside a test, `expect(actualValue)` is combined with a matcher such as `.toEqual()` or `.toHaveBeenCalledWith()` to assert expected behavior.

- `jest.mock('axios')` automatically replaces the entire axios module with an auto-mocked version, where every exported function becomes a `jest.fn()` mock function.
- `axios.get.mockResolvedValue(data)` configures the mocked get function to return a Promise that resolves with data, exactly like a successful real HTTP call would, but instantly and deterministically.
- `expect(axios.get).toHaveBeenCalledWith(url)` asserts that the mocked function was called with the expected argument, letting the test verify GitClient built the correct request URL.

```
import axios from 'axios';

jest.mock('axios'); // replace the whole axios module with mocks
axios.get.mockResolvedValue({ data: [...] }); // configure what it resolves to
```

Lab Steps

Step 1: Create the React application

```
npx create-react-app gitclientapp
cd gitclientapp
```

Open the gitclientapp folder in Visual Studio Code.

Step 2: Install axios

In the VS Code terminal, install the axios library used to call the GitHub API:

```
npm install axios
```

Step 3: Create the GitClient module

GitClient is a small class that wraps a single responsibility: given a GitHub username, fetch that user's repositories and return just their names. Keeping this logic in its own module (rather than inline inside a component) is exactly what makes it easy to unit test in isolation.

src/GitClient.js

```
import axios from 'axios';

class GitClient {
  constructor(username) {
    this.username = username;
    this.baseUrl = 'https://api.github.com';
  }

  getRepositories() {
    return axios
      .get(`${this.baseUrl}/users/${this.username}/repos`)
      .then((response) => response.data.map((repo) => repo.name));
  }
}

export default GitClient;
```

Step 4: Modify App.js to use GitClient

App creates a GitClient for the username 'techiesyed' and calls getRepositories() inside componentDidMount(), storing the result in state so it can be rendered as a list.

src/App.js

```
import React, { Component } from 'react';
import GitClient from './GitClient';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      repositories: [],
      loading: true,
      error: null
    };
    this.gitClient = new GitClient('techiesyed');
  }

  componentDidMount() {
    this.gitClient
      .getRepositories()
      .then((repositories) => {
        this.setState({ repositories, loading: false });
      })
      .catch((error) => {
        this.setState({ error: error.message, loading: false });
      });
  }

  render() {
    const { repositories, loading, error } = this.state;

    if (loading) return <p>Loading repositories...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
      <div>
```

```

    <h2>Repositories for techiesyed</h2>
    <ul>
      {repositories.map((repo, index) => (
        <li key={index}>{repo}</li>
      ))}
    </ul>
  </div>
);
}
}

export default App;

```

Step 5: Build and run the application

```
npm start
```

This opens the app at <http://localhost:3000>. After the real call to api.github.com resolves, the page lists the actual public repository names for the GitHub user techiesyed.

Step 6: Create the unit test file

Create `GitClient.test.js` in the `src` folder, alongside `GitClient.js`. It imports `axios` and `GitClient`, mocks `axios` so no real network call is made, and verifies that `getRepositories()` correctly extracts repository names from whatever data `axios` returns.

src/GitClient.test.js

```

import axios from 'axios';
import GitClient from './GitClient';

// Replace the entire axios module with Jest's auto-mock
jest.mock('axios');

describe('Git Client Tests', () => {
  test('should return repository names for techiesyed', () => {
    // Dummy data shaped like a real GitHub API response
    const dummyData = {
      data: [
        { name: 'react-hooks-demo' },
        { name: 'node-api-starter' },
        { name: 'jest-mocking-examples' }
      ]
    };

    // Configure the mocked axios.get to resolve with the dummy data
    // instead of making a real HTTP call to api.github.com
    axios.get.mockResolvedValue(dummyData);

    const gitClient = new GitClient('techiesyed');

    return gitClient.getRepositories().then((repositories) => {
      // GitClient should have transformed the mocked response
      // into a plain array of repository names
      expect(repositories).toEqual([
        'react-hooks-demo',
        'node-api-starter',

```

```
    'jest-mocking-examples'
  ]);

  // Confirms GitClient called axios with the expected URL,
  // without ever hitting the real GitHub API
  expect(axios.get).toHaveBeenCalledWith(
    'https://api.github.com/users/techiesyed/repos'
  );
});
});
});
});
```

Step 7: Run the tests

```
npm test
```

Create React App runs Jest in watch mode by default. Press 'a' to run all tests once if prompted, or run in CI mode to execute once and exit:

```
CI=true npm test
```

Expected Output

```
PASS  src/GitClient.test.js
  Git Client Tests
    ✓ should return repository names for techiesyed (4 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.2 s
Ran all test suites.
```

Because `axios.get` is mocked, this test passes instantly and consistently every time it runs, with no dependency on network access or the real state of the `techiesyed` GitHub account.

Summary

- Isolating a unit under test from its real dependencies (like a network API) makes tests fast, deterministic, and precise about what failed.
- Mocking replaces a dependency, such as `axios`, with a controllable fake that returns predetermined data instead of performing the real operation.
- Jest provides `jest.mock()` to auto-mock an entire module and `mockResolvedValue()/mockReturnValue()` to configure what a mock function returns.
- `describe()` groups related tests under a suite name, and `test()` defines an individual test case with its own `expect()` assertions.
- `toHaveBeenCalledWith()` lets a test assert exactly how a mocked function (like `axios.get`) was invoked, confirming `GitClient` built the correct request without ever making a real HTTP call.